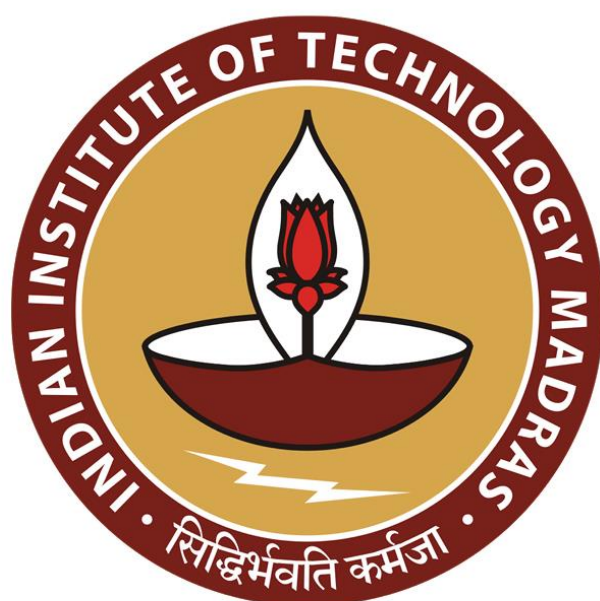




IIT Madras
ONLINE DEGREE

Programming Concepts Using Java
Online Degree Programme
B. Sc in Programming and Data Science
Diploma Level
Prof. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute

Lecture 6
Classes and Objects

So we saw some general properties of object oriented programming. So, let us explore concretely some things we have already seen in object oriented programming in python and explore why we need to be a little bit more elaborate about it and get to some of the things that we will study in java.

(Refer Slide Time: 00:31)

Programming with objects

- Object are like abstract datatypes
 - Hidden data with set of public operations
 - All interaction through operations — *messages, methods, member-functions, ...*
- Class
 - Template for a data type
 - How data is stored
 - How public functions manipulate data
- Object
 - Concrete instance of template
 - Each object maintains a separate copy of local data
 - Invoke methods on objects — send a message to the object

IIT Madras BSc Degree

Madhavan Mukund

Classes and objects

Programming Concepts using Java 2 / 11

So, remember that objects are like abstract data types. So, an object has a hidden set of data with a set of public operations and the interactions are through these operations which we sometimes call messages or methods or member functions. So, when we implement this in a programming language we have a two step process. So, first we have to do something which declares the properties of such an object.

So, this is what we usually call a class. So, class is a template for an abstract data type. So, it tells us what all data is stored inside the data type. So, it tells us the information about that private data typically and then it gives an implementation or a definition for

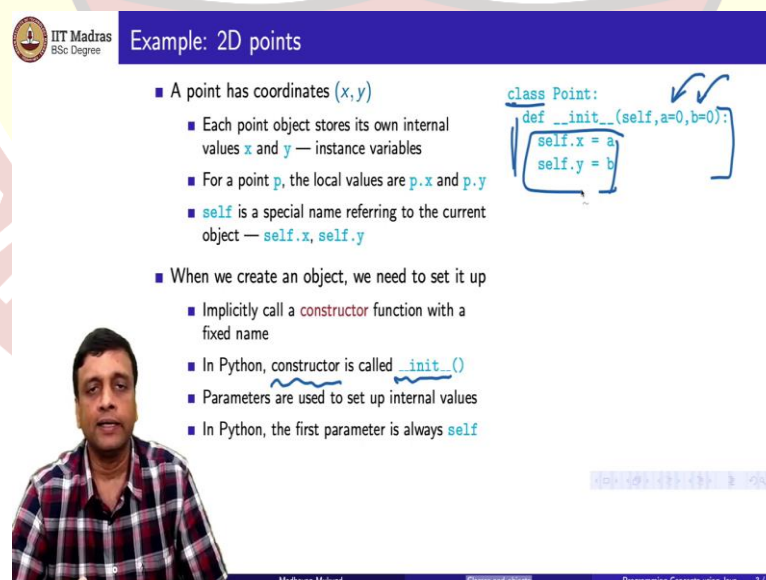
the functions that manipulate the data. So, this is the public interface and an object in this context is then a concrete instance of such a template.

So, we have a class for instance a class defining events and each event that we also use in our program is a concrete object of that type. So, it is like a class is like a type definition and an object is a variable of that type. So, each object will now have its own copy of the data that is described in the class. So, the class has some generic definition of the variables that are used and how they are initialized and how they are updated by the different functions.

And now each object will have its own copy of this data which is private to that object and will not interfere with any other object. So, we then invoke our function specifically with respect to a given object we tell this event this event simulate yourself or this graphical object draw yourself.

So, we invoked functions in the context of an object we do not pass data to functions we call an object and ask it to execute a function on itself. So, this is sometimes called sending a message to an object.

(Refer Slide Time: 02:25)



Example: 2D points

- A point has coordinates (x, y)
 - Each point object stores its own internal values x and y — instance variables
 - For a point p , the local values are $p.x$ and $p.y$
 - `self` is a special name referring to the current object — `self.x`, `self.y`
- When we create an object, we need to set it up
 - Implicitly call a **constructor** function with a fixed name
 - In Python, constructor is called `__init__()`
 - Parameters are used to set up internal values
 - In Python, the first parameter is always `self`

```
class Point:
    def __init__(self, a=0, b=0):
        self.x = a
        self.y = b
```

Mathavan Mahand Classes and objects Programming Concepts using Java 3 / 11

So, let us look at some examples in python and understand the basic ideas and also some of the limitations that a language like python has. So, here is a very simple example of an abstract data type that we would like to store it is a two dimensional point right it is a point on the plane which can be characterized by its x coordinate and y coordinate. So,

we need to first decide on how a point remembers where it is in the plane and it is natural that every point stores its own internal position x and y .

So, these are what are called instance variables. So, every instance of a point will have a separate copy of x and y which determines where that point is located on the plane. So, if I define p to be a point. So, p is a variable that is of type point then the instance variables with respect to p will be denoted usually $p \cdot x$ and $p \cdot y$. So, that I can distinguish between the x that belongs to point p and say point q so the point q will have values $q \cdot x$ and $q \cdot y$.

So, remember that we invoke these functions with respect to an object. So, we take a point and we say do this. So, for that point it has to talk about the x and y on itself right it does not have a name for itself in general because the name is defined outside. So, here we use a special name. So, in python usually we use the word `self` right. So, `self` is a name like p and q which refers to an object but refers to the object sitting inside. So, it's itself.

So, `self` is a natural way of referring to it. So, my value of x and my value of y are called `self.x` and `self.y`. So, here is a simple starting point for this point class in python. So, we use this word `class` to denote that we are starting with a class definition and now this is the first thing that you need because when I create an object I need to set it up I need to initialize the values of the variables.

So, normally remember that these values are supposed to be private they are supposed to be internal to the variables. So, I should not have a direct way of initializing it. So, then we call what is called a constructor. A constructor is a function that is implicitly called when we instantiate an object from a class we each time we create an object from a class we get a concrete variable pointing to a concrete object we have to set up the instance variables in that object and that is done through the constructor.

So, this is like a function which is called implicitly. So, since it is called implicitly it must have a fixed name. So, in the context of python this name is `__init__`. So, every time we create a point this function will be executed. So, this function takes arguments. So, these are the values that I would like

to provide the point and internally it sets the instance variables based on the arguments that I have provided.

So, this is how a point is set up or an object in general is set up by calling a constructor a constructor is a special function which is invoked implicitly when the function point or the object is created. You do not later on go and call this explicitly it's not a function that I can ask for later on it is just something that is implicitly done at the time of creation of the object.

(Refer Slide Time: 05:43)

IIT Madras BSc Degree

Adding methods to a class

- Translation: shift a point by $(\Delta x, \Delta y)$
 - $(x, y) \mapsto (x + \Delta x, y + \Delta y)$
 - Update instance variables
- Distance from the origin
 - $d = \sqrt{x^2 + y^2}$
 - Does not update instance variables
 - state of object is unchanged

(Hand-drawn diagram showing a point P at coordinates (x, y) in a 2D coordinate system. A right triangle is formed with the origin, with sides of length x and y, and a hypotenuse of length d. The point P is labeled (x, y).)

```
class Point:
    def __init__(self, a=0, b=0):
        self.x = a
        self.y = b

    def translate(self, dx, dy):
        self.x += dx
        self.y += dy

    def odistance(self):
        import math
        d = math.sqrt(self.x*self.x + self.y*self.y)
        return(d)
```

Madhavan Mukund | Classes and objects | Programming Concepts using Java | 4/11

So, having now created this object there will be more useful functions that manipulate it. So, in the case of a point one possibility is that we take a point which is sitting at some coordinate x y and we want to move it. So, we want to move it by certain amount which is given by Δx and Δy . So, we want to say move Δx steps in the x direction and move Δy steps in the y direction.

So, the new coordinate is going to be x plus Δx and y plus Δy . So, here is a function which does this translation as it is called in geometry. So, we translate this point by an amount dx and dy . So, dx stands for the Δx and dy of the Δy . So, naturally what we do is we update the instance variable. So, we have a `self.x` and the `self.y` which currently hold the values corresponding to position of the point now we want to now make x into x plus d dot x .

So, that is what this does this is x equal to x plus dx and y equal to y self dot y plus dy . So, this is a method that we now associate with this class. So, every point will now have a translate function which will take the instance variables of that point and translate it by the amount specified you could also have functions which do not change the state of the variable.

So, the state of an object is really telling us what are all the current values of the instant variables. So, translation affects the state it moves a point from one place to another place. So, it is no longer the same point but if I ask some property of the point for instance I ask for its distance from the origin. So, if I draw this thing and I have a point p here then it is at x y right then by using Pythagoras theorem I know that if this is x and this is y then the distance is the hypotenuse which is the square root of x squared plus y squared.

So, this is the function `o distance` the distance from the origin. So, it does not take any argument other than this mandatory argument `self`. So, python has this mandatory argument `self` which is there in every function which is defined in this object oriented way. So, we will see that java does not need this. So, you have the `self` but you have no other argument.

So when I call it I just say `o distance` and what does `o distance` do it looks up the current value of `self dot x` and `self dot y` and it performs this calculation it takes square root of x squared plus y squared and it returns d without changing x and y . So, this is just a reporting function is telling us some property about the point without updating its state.

(Refer Slide Time: 08:10)

■ Polar coordinates: (r, θ) , not (x, y)

■ $r = \sqrt{x^2 + y^2}$

■ $\theta = \tan^{-1}(y/x)$

■ Distance from origin is just r

■ Translation

■ Convert (r, θ) to (x, y)

■ $x = r \cos \theta, y = r \sin \theta$

■ Recompute r, θ from $(x + \Delta x, y + \Delta y)$

■ Interface has not changed

■ User need not be aware whether representation is (x, y) or (r, θ)

```
def translate(self, dx, dy):
    x = self.r * math.cos(self.theta)
    y = self.r * math.sin(self.theta)
    x += dx
    y += dy
    self.r = math.sqrt(x*x + y*y)
    if x == 0:
        self.theta = math.pi/2
    else:
        self.theta = math.atan(y/x)
```



Now the whole point of having this internal implementation different from the public function is that we can change the internal implementation and keep the public interface untouched as far as the user is concerned. Of course the way the public interface works will change but the user will not see any difference in terms of the answers that he gets. So, here for instance we could take x, y . So, this point x, y and instead represent it in terms of its distance r and the angle θ .

So, this is what is called the polar representation and how do you get the polar representation from these. So, called rectangular representation well clearly r is the same as a distance. So, r^2 is equal to $x^2 + y^2$ and we know that from trigonometry that $\tan \theta$ is opposite over adjacent y by x . So, from this you can say that θ is a $\tan$ inverse of y by x and r is the square root of $x^2 + y^2$.

So, this is how we are going to represent it internally but externally nothing is changed. So, from the point of view of this constructor that the person sees they will still be giving x and y as the inputs. So, for the outside world the point is still working in x, y coordinates. So, internally when I pass it a value a and a value b I calculate the value of r using this formula and I set the instance variable r self dot r .

Similarly I said self dot θ now there is a problem with self dot θ which is that when I am at 90 degrees right y by x and x is 0 then this quantity will not work. So, even though I know the angle is 90 degrees I cannot use $\tan$ inverse because the division of y by x will be an error. So, I check explicitly that if the a coordinate that is the x

coordinate of the point is 0 that it lies on the y axis then I explicitly set theta to be $\pi/2$ which is 90 degrees.

Otherwise I set it to be the tan inverse of b by a. So, what we have done here is we have changed the internal representation but the same constructor works. So, the user is not even aware in some sense that internally instead of x and y the instance variables have changed to r and theta and likewise we can now take the functions that we have as a public interface of this and re-implement them. So, remember this idea that if I do a data refinement.

If I change the way the data is stored the functions that use the data must also change but in this object oriented thing the functions are sitting with the data. So, I know that if I change this part I must also change this part. So, now the distance from the origin for instance as we said is just r because that's exactly the same way we calculate r it is the distance from the origin to the point is precisely the definition of r.

So, we just have to return self dot or no calculation is required. However if we do translation we have to do some work because remember from the external user of the point is still working in these x y coordinates. So, if I want to go from x, y to x plus delta x, y plus delta y this is what translate does but internally I am working in r theta. So, what I have to do is I have to convert first the r theta to x, y then I have to go from x, y to x plus delta x, y plus delta y and then come back to r theta like we did in the initial constructor.

So, how do we go the other way well we use again the trigonometry that we know that x is $r \cos \theta$ and y is $r \sin \theta$. So, the first thing we do is that we set up. So, this is not an instance variable. So, it is not self dot text self dot why it is just a typical intermediate variable that you would use in any function is just some vanilla x and vanilla y which we are using to keep track of temporarily.

So, we temporarily calculate x and y from self dot r and self dot theta then we update that x using the translated parameters and now we go back and do what we did in the constructor. So, we reset self dot r based on the new x and the new y and we reset theta based on the new x and y and once again we take care that if the new x has become 0 if

the translated point lies on the y axis we directly set the angle to 90 otherwise we use tan inverse of y by x.

So, again from an external perspective it is the same effect as taking x, y and translating it by a certain amount in x upon y but internally now the way we do this manipulation has changed because the instance variables have moved from x, y to r theta. And the fact that this has to be done internally that you have to make a change is transparent because inside the class definition we know that every function will now have to respect the new instance variables.

So, the data refinement goes hand in hand with this code refinement. So, the user is not aware in principle as to which representation we are using and we could be using even another representation the user really does not care. As long as from the users perspective these points behave like points at x, y which can be moved around like normal rectangular points with Cartesian coordinates the user is happy.

(Refer Slide Time: 13:25)

Abstraction

- User should not know whether `Point` uses `(x,y)` or `(r,theta)`
 - Interface remains identical
 - Even constructor is the same
- Python allows direct access to instance variables from outside the class
 - `p = Point(5,7)`
 - `p.x = 4` # Point is now (4,7)
 - Breaks the abstraction
 - Changing the internal implementation of `Point` can have impact on other code
 - Rely on programmer discipline

```
class Point:
    def __init__(self,a=0,b=0):
        self.x = a
        self.y = b

class Point:
    def __init__(self,a=0,b=0):
        self.r = math.sqrt(a*a + b*b)
        if a == 0:
            self.theta = math.pi/2
        else:
            self.theta = math.atan(b/a)
```

Madhavan Mukund

Classes and objects

Programming Concepts using Java 6/11

So, from the purpose of abstraction what we claimed is the user should not know whether the point uses x, y or r theta that these should be actually private variables of this class which are not accessible outside. But unfortunately in python it is not possible to actually declare this to be private. So, supposing we start by creating a point at 5, 7. So, we call this point class with parameter 5, 7.

So, now `p` points to a point at coordinates 5, 7. Now of course we could translate it to 4, 7 by applying the translate operator with -1 and 0 move the `x` by -1 move the `y` by 0 but python will also allow you to directly take `p` and replace `p.x` by 4. So, this will of course work if we are in this row right if we are working in this mode when we have an internal `x` and internal `y` this assignment by python will be legal.

If we are working in this mode then this will just create an extra instance variable which is not even part of the class definition. So, there is already a `p.r` and the `p.theta` and it is just created a `p.x` and set it to 4. So, this kind of direct assignment to these instance variables from outside the class breaks this abstraction and this means that tomorrow if I do change the implementation from `x, y` to `r, theta` then this code will now suddenly stop working.

And the person who has written this code will have no idea that this is going to stop working until they actually hit a runtime bug. So, this is what we want to avoid we want to avoid that the functionality of these objects that we are using from outside changes because of some change to that object we would like the interface the public interface to remain unchanged.

So, to avoid this we really want to explore the; I mean we want to insist that programmers will not directly do this. So, you can have some kind of a coding manual or a discipline for programming in your team which says you never touch an instance variable but this really is relying on the good behaviour of a programmer. And the problem with code is that it tends to get maintained over a long lifetime across a number of different groups of people.

So, each of them may have their own assumptions about what is allowed and what is not allowed. So, if you leave it to the good taste and the good sense of the programmer very often mistakes will arise just because of a misunderstanding of what is expected.

(Refer Slide Time: 15:51)



- Define `Square` to be a subtype of `Rectangle`
 - Different constructor
 - Same instance variables

```
class Rectangle:
    def __init__(self, w=0, h=0):
        self.width = w
        self.height = h

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

class Square(Rectangle):
    def __init__(self, s=0):
        self.width = s
        self.height = s
```



Madhavan Mukund

Classes and objects

Programming Concepts using Java

7/11

So, let us also look at this idea of subtyping and inheritance in the context of python. So, we saw earlier a very simple single point x, y. So, what if we want to think of a rectangle which has two dimensions a width and height. So, we have a class for rectangle which has two internal variables self dot width and self dot height. And with respect to these two variables it has two public functions one which is the area which is this the width times the height and the other is the perimeter which is two times.

So, it is the width plus the width plus the height plus the height. So, it is two times the width plus height right. So, these are very standard quantities at any anytime in school you learn about rectangles. Now what is a square a square is a rectangle in which the width is equal to the height. So, we can now define a square to be a sub type of rectangle and the way in python you do this is that in the class definition of square you put the type rectangle in the bracket.

So, normally when you say class rectangle you do not say anything but if you have want to say that square is a sub type of rectangle you put the parent type in the bracket. So, now this says that square is a special case of a rectangle and here I can define a new constructor because if I have a square I only have one quantity I have the side I do not have a width and a height.

So, I want to say that I take one value to construct a square namely the side of the square and I will reset the width and the height both to be that value. So, I pass only 1 s and I explicitly set self load width and self dot height. So, the constructor for square as I have

written it here is slightly different from there because there we got two parameters and we set them here I get one parameter in a second.

(Refer Slide Time: 17:44)

Subtyping and inheritance

- Define `Square` to be a subtype of `Rectangle`
 - Different constructor
 - Same instance variables
- The following is legal

```
s = Square(5)
a = s.area()
p = s.perimeter()
```
- `Square` inherits definitions of `area()` and `perimeter()` from `Rectangle`

```
class Rectangle:
    def __init__(self, w=0, h=0):
        self.width = w
        self.height = h

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

class Square(Rectangle):
    def __init__(self, s=0):
        self.width = s
        self.height = s
```

No area()

Madhavan Mukund

Now what is the property of subtyping and inheritance? Well if I now start by saying that I want a square with side 5 this will create implicitly an object which has width 5 and height 5 because that is what is happening here. And with this now if I ask for the area of the square I will get a legitimate value of 25 even though inside this there is no area function here. So, this area is actually using that area.

So, this is the idea of inheritance. So, because square is a sub type of rectangle the functions which are defined inside rectangle are available to square. So, I do not have to redefine them unless I want to change them. So, if I do not have any need to change them if they work the same because I have set up these instance variables to mean the same things then I can go ahead and reuse it. So, perimeter and area are both inherited by square from rectangle. So this so far so good.

(Refer Slide Time: 18:40)



- Can change the instance variable in Square

- self.side

- The following gives a run-time error

```
s = Square(5)
a = s.area()
p = s.perimeter()
```

- Square inherits definitions of area() and perimeter() from Rectangle

- But s.width and s.height have not been defined!

- Subtype is not forced to be an extension of the parent type

```
class Rectangle:
    def __init__(self, w=0, h=0):
        self.width = w
        self.height = h

    def area(self):
        return self.width*self.height

    def perimeter(self):
        return 2*(self.width+self.height)
```

```
class Square(Rectangle):
    def __init__(self, s=0):
        self.side = s
```



Madhavan Mukund

Classes and objects

Programming Concepts using Java

8 / 11

Now supposing I decide that oh in a square I do not need to keep track of two separate quantities there is only one side. So, let me change the instance variable in square to be just self.taught side. So, I just remember the side I do not set the width and height appropriately. Now if I try the same code which earlier work due to inheritance. These two functions will now at run time they will fail.

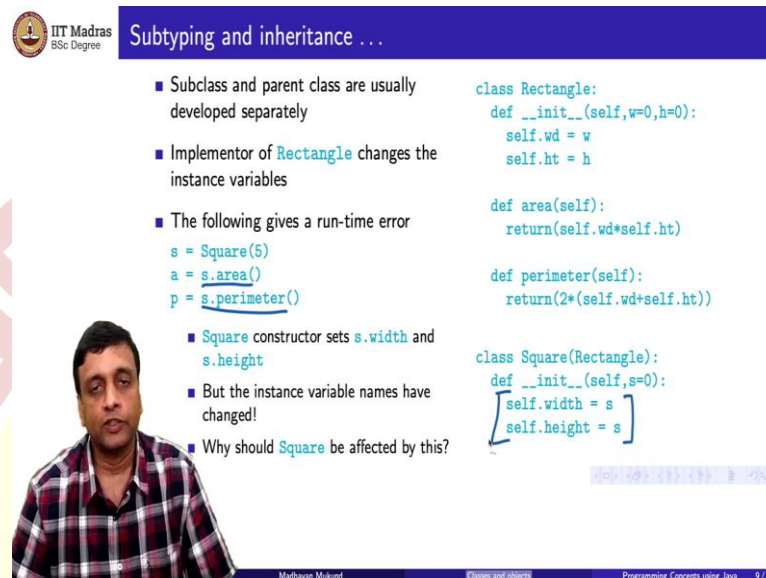
That is compile time they should exist. So, statically these functions are available because square has been defined to be a sub type of rectangle. So, those functions are available to square. So, if they are not defined in square and if I say s dot area python has no way to know before running the code that this s dot area will not work. But when it actually comes to executing s dot area it sees that it needs these quantities self dot width and self dot height.

And unfortunately in our square code we change the instance variable to we just self.taught side but we did not update self dot width or self dot height explicitly. And since we did not update them they are as good as undefined variables. So, what we will end up here with both area and perimeter is a name error for the values that a perimeter and area are used for their calculations because self.s dot width and s dot height have not been defined.

So, the sub type is not forced in some sense to be a proper extension of the parent type. Normally we say that the subtype can do everything the parent type will do and more and in particular that means that it should have all the instance variables that the parent type

would have declared and more because the functions would use these instance variables. But here in python we are able to do this kind of crazy thing where we can set up instance variables in the sub type and omit some instance variables from the parent type. So, this is clearly not a good thing.

(Refer Slide Time: 20:27)



IIT Madras
BSc Degree

Subtyping and inheritance ...

- Subclass and parent class are usually developed separately
- Implementor of `Rectangle` changes the instance variables
- The following gives a run-time error


```
s = Square(5)
a = s.area()
p = s.perimeter()
```
- `Square` constructor sets `s.width` and `s.height`
- But the instance variable names have changed!
- Why should `Square` be affected by this?

```
class Rectangle:
    def __init__(self,w=0,h=0):
        self.wd = w
        self.ht = h

    def area(self):
        return(self.wd*self.ht)

    def perimeter(self):
        return(2*(self.wd+self.ht))

class Square(Rectangle):
    def __init__(self,s=0):
        [self.width = s
         self.height = s]
```

Madhavan Mukund

Classes and objects

Programming Concepts using Java

9 / 11

The other problem with this kind of notation that python allows is that it is not usually the case that you extend your own type. So, normally there is a library of objects which is available in the language either because it is there in the library as part of the programming language itself or as over time some people have developed some objects which are generally useful for people to use.

Now you have a specific piece of code to write and you want to extend it. So, you are using sub class of a given class but you do not have access to or control over the parent class. So, the person who writes the sub class is not expected to be the developer of the parent class. So, when you use the parent class you are using it like everybody else right as a kind of encapsulated type of which you can only use the public features.

So, this is the earlier working example that we had for rectangle and square where the square actually set the instance variables correctly. Now tomorrow the maintainer of rectangle decides that he is tired of writing width and height as a full word. So, he changes these names to `wd` and `ht` and correspondingly changes definitions of `area` and `perimeter`. So, as far as rectangles go this consistent change has been made.

So, everything works. But now again if we go back to our earlier code which should work the inherited code once we call `s dot area` and `s dot perimeter` the problem here is that the writer of square assumed that the instance variables that rectangle uses are called width and height and has not been notified or is not aware of the fact they have actually changed their name.

So, in order to write a consistent subclass square it appears that you actually have to know something about the private implementation of rectangle and it is not clear why this should be the case.

(Refer Slide Time: 22:23)

IIT Madras
BSc Degree

Subtyping and inheritance ...

- Need a mechanism to hide private implementation details
- Declare component private or public

```
class Rectangle:
    def __init__(self, w=0, h=0):
        self.wd = w
        self.ht = h

    def area(self):
        return(self.wd*self.ht)

    def perimeter(self):
        return(2*(self.wd+self.ht))

class Square(Rectangle):
    def __init__(self, s=0):
        self.width = s
        self.height = s
```

Mathavan Mahand

Classes and objects

Programming Concepts using Java 10/11

So, the main thing that python lacks is a way to first of all specify that something is private and something else is public. So, we need declarations which python does not have at all. So, the first type of declaration that python does not have is whether something is in into a float or a char or something else. And now we also need a kind of orthogonal thing it is not the type of the variable but the visibility of the variable.

We need to know whether this thing is private only available within the class to functions like this. So, for example these two values can be used only by functions which are defined within the class because they are part of the whole implementation and they should not be visible outside this is what we want.

(Refer Slide Time: 23:10)



- Need a mechanism to hide private implementation details
 - Declare component private or public
- Working within privacy constraints
 - Instance variables `wd` and `ht` of `Rectangle` are private
 - How can the constructor for `Square` set these private variables?
 - `Square` does (and should) not know the names of the private instance variables
- Need to have elaborate declarations
 - Type and visibility of variables

static type checking catches errors early

```
class Rectangle:
    def __init__(self, w=0, h=0):
        self.wd = w
        self.ht = h

    def area(self):
        return(self.wd*self.ht)

    def perimeter(self):
        return(2*(self.wd+self.ht))

class Square(Rectangle):
    def __init__(self, s=0):
        self.width = s
        self.height = s
```



But then if these are indeed private then how does this square set itself up. So, square needs to be able to set the instance variables of a rectangle but square is not allowed to right it should not. So, it does not and it should not know the names of the private variables of rectangle. So, we need to have a lot more things. So, it is not just the typing and the declaration which we have to do but we also have to provide now mechanisms additional mechanisms which allow us to work within these privacy constraints.

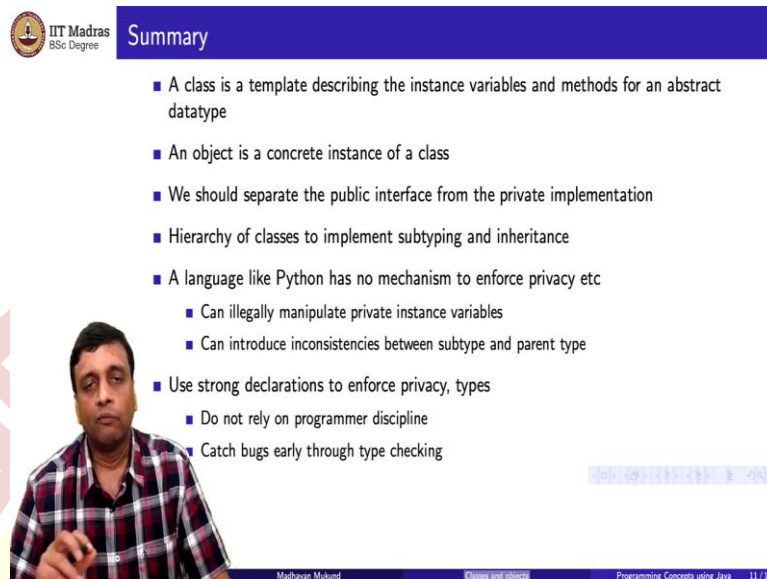
So, these privacy constraints impose a certain discipline on how these variables are used but they also impose a restriction on how they can be set up. So, we need to provide mechanisms to legally bypass these restrictions and set up these variables. So, how will square set these two values without knowing what they are. So, we need to have more elaborate declarations which declare both the type and the visibility.

So, that is one of the things that we will be seeing now. So, java will turn out to be a much more complicated language than python but a lot of the complex syntax that comes into java is forced because of having to elaborate all these aspects more explicitly. So, we need to have all these different types of things like privacy and all that well defined. So that things are used in a proper manner.

And the main goal of this is again to make sure that we can do static type checking. So, we want to be sure that if we are using a sub type of a parent type that the sub type will work properly in all contexts where we expect the parent type to work which it was not

doing because sometimes the subtype in our case was not defining the instance variables properly.

(Refer Slide Time: 24:53)



Summary

- A class is a template describing the instance variables and methods for an abstract datatype
- An object is a concrete instance of a class
- We should separate the public interface from the private implementation
- Hierarchy of classes to implement subtyping and inheritance
- A language like Python has no mechanism to enforce privacy etc
 - Can illegally manipulate private instance variables
 - Can introduce inconsistencies between subtype and parent type
- Use strong declarations to enforce privacy, types
 - Do not rely on programmer discipline
 - Catch bugs early through type checking

Madhavan Mukund Classes and objects Programming Concepts using Java 11 / 11

So, to summarize when we go to actually implementing this idea of object-oriented programming we start with a class. So, class is a template and a template describes the variables and the functions which are going to be available for an abstract data type and an object is a concrete instance of this template. So, an object is an actual variable which has place to store these local things and so on.

And we would like this definition to clearly separate the public interface from the private implementation. And we would also like these classes to be organized in a hierarchy to allow subtyping and inheritance. So, what we saw that is that python has a mechanism to define classes but it really is very limited in enforcing these ideas of abstraction. So, we can illegally manipulate private variables because there is no concept of privacy in python.

And we can also unintentionally introduce inconsistencies between subtypes and parent types because there is no obligation that a subtype must have the same instance variables defined as a parent type. So, ideally what we need is a mechanism to have these strong declarations to enforce privacy and types. So, that we can move away from relying on the discipline of the programmer to make things work properly and instead allow the compiler to check these things statically and catch bugs early through type checking.